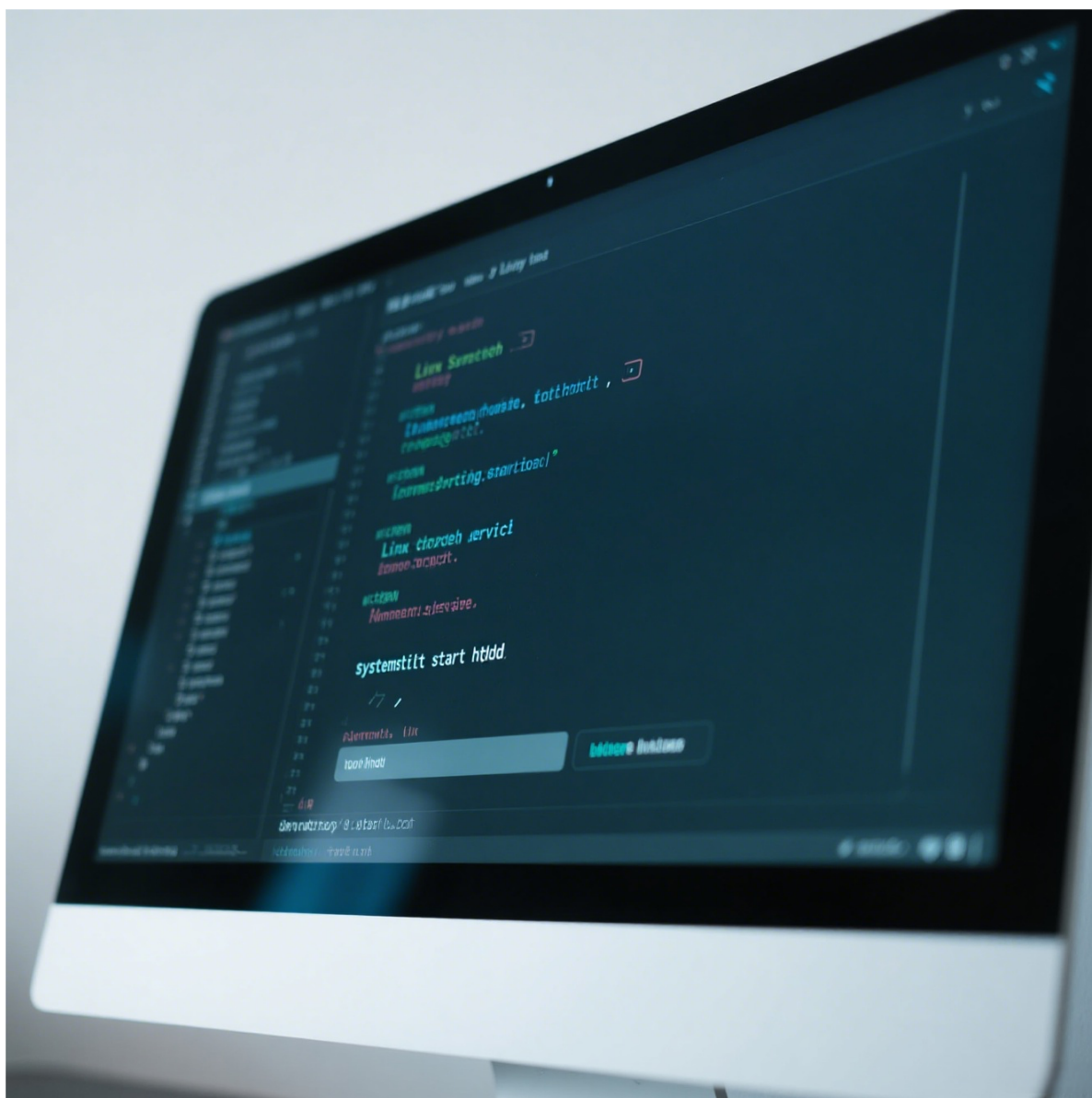


Linux的系统管理



一、进程管理

什么是进程？

进程是正在执行的一个程序或命令，每一个进程都是一个运行的实体，都有自己的地址空间，并占用一定的系统资源。

学习进程管理目标

- 判断服务器健康状态
- 查看系统中所有进程
- 杀死进程

(1.1) 进程查看

使用下面命令查看系统所有进程

```
ps aux #查看系统中所有进程，使用BSD操作系统格式，建议使用这个格式
#ps aux --sort=-%cpu
ps -lef#查看系统中所有进程，使用Linux标准命令格式
```

```
[root@theon ~]# ps aux
USER          PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root           1  0.1  0.3 128016 6544 ?        Ss   15:20   0:01 /usr/lib/systemd/systemd --switched-root --system --deserialize 22
root           2  0.0  0.0      0     0 ?        S    15:20   0:00 [kthreadd]
root           3  0.0  0.0      0     0 ?        S    15:20   0:00 [ksoftirqd/0]
root           5  0.0  0.0      0     0 ?        S<   15:20   0:00 [kworker/0:0H]
root           7  0.0  0.0      0     0 ?        S    15:20   0:00 [migration/0]
root           8  0.0  0.0      0     0 ?        S    15:20   0:00 [rcu_bh]
root           9  0.0  0.0      0     0 ?        R    15:20   0:00 [rcu_sched]
root          10  0.0  0.0      0     0 ?        S<   15:20   0:00 [lru-add-drain]
root          11  0.0  0.0      0     0 ?        S    15:20   0:00 [watchdog/0]
root          13  0.0  0.0      0     0 ?        S    15:20   0:00 [kdevtmpfs]
root          14  0.0  0.0      0     0 ?        S<   15:20   0:00 [netns]
root          15  0.0  0.0      0     0 ?        S    15:20   0:00 [khungtaskd]
root          16  0.0  0.0      0     0 ?        S<   15:20   0:00 [writeback]
root          17  0.0  0.0      0     0 ?        S<   15:20   0:00 [kintegrityd]
root          18  0.0  0.0      0     0 ?        S<   15:20   0:00 [bioset]
root          19  0.0  0.0      0     0 ?        S<   15:20   0:00 [kblockd]
root          20  0.0  0.0      0     0 ?        S<   15:20   0:00 [md]
```

各个属性说明

- ✧ USER: 该进程是由哪个用户产生的;
- ✧ PID: 进程的ID号;
- ✧ %CPU: 该进程占用CPU资源的百分比, 占用越高, 进程越耗费资源;
- ✧ %MEM: 该进程占用物理内存的百分比, 占用越高, 进程越耗费资源;
- ✧ VSZ: 该进程占用虚拟内存的大小, 单位KB;
- ✧ RSS: 该进程占用实际物理内存的大小, 单位KB;
- ✧ TTY: 该进程是在哪个终端中运行的。其中tty1-tty7代表本地控制台终端, tty1-tty6是本地的字符界面终端, tty7是图形终端。pts/0-256代表虚拟终端。

- ✧ STAT: 进程状态。常见的状态有: R: 运行、S: 睡眠、T: 停止状态、s: 包含子进程、+: 位于后台
- ✧ START: 该进程的启动时间
- ✧ TIME: 该进程占用CPU的运算时间, 注意不是系统时间
- ✧ COMMAND: 产生此进程的命令名

查看进程树

使用命令 `pstree` 查看进程和子进程

命令格式

`ps tree` [选项] [进程号(可选)]

选项:

-p 显示进程PID

-u 显示进程的所属用户

最小化安装没有ps tree命令, 需要yum -y install ps misc 进行安装

如果要看到指定进程号的进程树信息, 可以先使用pgrep获取进程的进程PID, 然后 ps tree -p [PID]进行查看

(1.2) 查看系统健康状态

使用 `top` 命令进行查看, 需要注意的是, 执行top命令后, 会进入交互界面

`top` [选项]

-d 秒数: 指定top命令每隔几秒更新。默认是3秒

在top命令的交互模式当中可以执行的命令:

?或h: 显示交互模式的帮助

P: 以CPU使用率排序

M: 以内存的使用率排序

N: 以PID排序

q: 退出top

如果需要非交互执行top, 可以top -b -n 2 组合, 表示非交互每隔3秒得出2次结果

命令执行结果如下:

```
top - 15:44:02 up 23 min, 1 user, load average: 0.00, 0.01, 0.03
Tasks: 93 total, 2 running, 91 sleeping, 0 stopped, 0 zombie
%Cpu(s): 0.3 us, 0.0 sy, 0.0 ni, 99.7 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st
KiB Mem : 1865280 total, 1551572 free, 143080 used, 170628 buff/cache
KiB Swap: 2097148 total, 2097148 free, 0 used. 1539296 avail Mem

  PID USER      PR  NI   VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
    1 root        20   0 128016   6548   4152 S   0.0   0.4   0:01.11 systemd
    2 root        20   0     0      0      0 S   0.0   0.0   0:00.00 kthreadd
    3 root        20   0     0      0      0 S   0.0   0.0   0:00.05 ksoftirqd/0
    5 root         0 -20     0      0      0 S   0.0   0.0   0:00.00 kworker/0:0H
    7 root        rt    0     0      0      0 S   0.0   0.0   0:00.00 migration/0
    8 root        20   0     0      0      0 S   0.0   0.0   0:00.00 rcu_bh
    9 root        20   0     0      0      0 S   0.0   0.0   0:00.35 rcu_sched
```

命令分析-----主要看前5行:

第一行信息为任务队列信息

内容	说明
12:26:46	系统当前时间
up 1 day, 13:32	系统的运行时间，本机已经运行1天13小时32分钟
2 users	当前登录了两个用户
load average: 0.00, 0.00, 0.00	系统在之前1分钟，5分钟，15分钟的平均负载。一般认为小于1时，负载较小。如果大于1，系统已经超出负荷。 与cpu核数有关

第二行为进程信息

内容	说明
Tasks: 95 total	系统中的进程总数
1 running	正在运行的进程数
94 sleeping	睡眠的进程
0 stopped	正在停止的进程
0 zombie	僵尸进程。如果不是0，需要手工检查僵尸进程 正在终止，但还未终止的进程

第三行为CPU信息

内容	说明
Cpu(s): 0.1%us	用户模式占用的CPU百分比
0.1%sy	系统模式占用的CPU百分比
0.0%ni	改变过优先级的用户进程占用的CPU百分比
99.7%id	空闲CPU的CPU百分比
0.1%wa	等待输入/输出的进程的占用CPU百分比
0.0%hi	硬中断请求服务占用的CPU百分比
0.1%si	软中断请求服务占用的CPU百分比
0.0%st	st (Steal time) 虚拟时间百分比。就是当有虚拟机时，虚拟CPU等待实际CPU的时间百分比。

主要看这个

第四行为物理内存信息

内容	说明
Mem: 625344k total	物理内存的总量，单位KB
571504k used	已经使用的物理内存数量
53840k free	空闲的物理内存数量，我们使用的是虚拟机，总共只分配了628MB内存，所以只有53MB的空闲内存了
65800k buffers	作为缓冲的内存数量

第五行为交换分区 (swap) 信息

内容	说明
Swap: 524280k total	交换分区（虚拟内存）的总大小
0k used	已经使用的交互分区的大小
524280k free	空闲交换分区的大小
409280k cached	作为缓存的交互分区的大小

top交互中的其他命令

u/U：指定显示用户

```
top - 17:15:41 up 1:55, 1 user, load average: 0.00, 0.01, 0.03
Tasks: 93 total, 1 running, 92 sleeping, 0 stopped, 0 zombie
%Cpu(s): 0.0 us, 0.0 sy, 0.0 ni, 100.0 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st
KiB Mem : 1865280 total, 1498660 free, 166652 used, 199968 buff/cache
KiB Swap: 209/148 total, 209/148 free, 0 used. 1498708 avail Mem
Which user (blank for all) postfix
PTD USER PR NT VIRT RES SHR S %CPU %MEM TIME+ COMMAND
1 root 20 0 128016 6548 4152 S 0.0 0.4 0:01.17 systemd
2 root 20 0 0 0 0 S 0.0 0.0 0:00.00 kthreadd
3 root 20 0 0 0 0 S 0.0 0.0 0:00.07 ksoftirqd/0
```

(1.3) 虚拟文件系统/proc

/proc是内存中有关系统进程的信息，也有系统资源相关的信息

例如

```
pgrep sshd
#输出结果是981和1445
ls /proc/1445
```

```
[root@theon ~]# ls /proc/1445
attr          clear_refs    cpuset        fd            limits        mem           net           oom_score     personality   schedstat     stack         syscall       wchan
autogroup     cmdline       cwd           fdinfo        loginuid      mountinfo     ns            oom_score_adj projid_map    sessionid     stat          task
auxv          comm          environ       gid_map       map_files     mounts        numa_maps     pagemap       root          setgroups    statm         timers
cgroup        coredump_filter exe           io            maps          mountstats    oom_adj       patch_state   sched         smaps        status        uid_map
```

对于安全分析来说，比较有价值的文件/目录

- `cmdline`: 进程启动时的完整命令行参数。如果命令行中包含密码、API 密钥或其他敏感凭证，这会直接暴露它们。即使没有敏感信息，了解进程启动方式也有助于攻击者了解系统配置或识别潜在的漏洞。

- `environ`: 进程的环境变量。和 `cmdline` 类似，环境变量也可能包含敏感信息，如数据库连接字符串、加密密钥、临时凭证等。
- `exe`: 指向进程的可执行文件的符号链接。这个文件可以告诉攻击者进程运行的是哪个程序。如果一个合法的程序被恶意替换，`exe` 可以帮助识别。
- `fd/ (目录)`: 包含了进程打开的所有文件描述符的符号链接。通过查看这些链接，攻击者可以了解进程正在访问哪些文件、设备或网络连接。这可能揭示敏感文件路径、进程间的通信方式，甚至允许攻击者劫持文件描述符进行数据窃取或篡改。
- `maps / smaps`: 进程的内存映射。这些文件详细列出了进程的内存区域、权限（读/写/执行）以及对应的文件（如果是文件映射）。`smaps` 提供更详细的内存使用信息，包括私有内存、共享内存等，有助于更深入地分析进程的内存布局。进程注入（Process Injection）通常会利用这些信息来寻找合适的内存区域注入恶意代码。

注: 一些进程信息查看的工具（如 `lsOf`）都是读取 `/proc` 下进程目录里面的文件或目录信息后，进行汇总输出

扩展: `/proc` 目录中也有 CPU 和内存的详细信息

查看当前 CPU 信息

```
cat /proc/cpuinfo
```

```
[root@theon ~]# cat /proc/cpuinfo
processor       : 0
vendor_id      : GenuineIntel
cpu family     : 6
model          : 141
model name     : 11th Gen Intel(R) Core(TM) i5-11400H @ 2.70GHz
stepping       : 1
microcode      : 0x34
cpu MHz        : 2687.999
cache size     : 12288 KB
physical id    : 0
siblings       : 1
core id        : 0
cpu cores      : 1
apicid         : 0
initial apicid : 0
fpu            : yes
fpu_exception  : yes
cpuid level    : 27
wp             : yes
flags          : fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat pse36
rch_perfmon rep_good nopl xtopology tsc_reliable nonstop_tsc eagerfpu pni pclmulqdq ssse3
save_ux f16c rdrand hypervisor lahf_lm abm 3dnowprefetch fsgsbase tsc_adjust bmi1 avx2 s
opt clwb avx512cd sha_ni avx512bw avx512vl xsaveopt xsavec xgetbv1 ibpb ibrs stibp arat a
ies
bogomips       : 5375.99
clflush size   : 64
cache_alignme  : 64
address sizes  : 45 bits physical, 48 bits virtual
power managem  :
```

查看当前内存的信息

```
cat /proc/meminfo|more
```

```
[root@theon ~]# cat /proc/meminfo|more
MemTotal:      1865280 kB
MemFree:       1329216 kB
MemAvailable:  1495048 kB
Buffers:       2108 kB
Cached:        283016 kB
SwapCached:    0 kB
Active:        201064 kB
Inactive:      156772 kB
Active(anon):  73104 kB
Inactive(anon): 9300 kB
Active(file):  127960 kB
Inactive(file): 147472 kB
Unevictable:   0 kB
Mlocked:      0 kB
SwapTotal:     2097148 kB
SwapFree:      2097148 kB
Dirty:         0 kB
Writeback:     0 kB
AnonPages:     72728 kB
Mapped:        26660 kB
Shmem:         9692 kB
Slab:          84160 kB
```

(1.4) 进程优先级调整

1.4.1 进程优先级定义

Linux系统Max进程数1000-----2000

一个进程正好有40种不同级别的NICE值，值越高，优先级越低

优先级取值范围为（-20,19），越小优先级越高, 默认优先级是0

系统允许ROOT用户设置负nice级别以及降低现有进程的nice级别；对普通非特权用户仅允许设置正的nice级别，此外，他们只能对现有进程提升nice级别。而不能降低nice级别

注意：在linux中除了nice级别以外，有很多方法可以影响到进程优先级和资源使用情况，比如备用的调度程序策略和设置、控制组等，但是nice级别是里面最易于使用的一种方法，而且它不仅系统管理员可以使用，普通用户也可以使用

1.4.2 进程优先级的调整

方法一：使用命令调整

命令1：

```
nice -n [-20~19] [命令]
```

作用:**指定**程序的运行优先级

范例:


```
nice -n -10 vi a.txt
#指定此次vim进程的优先级为-10
```

```
top - 17:55:25 up 2:55, 2 users, load average: 0.00, 0.02, 0.04
Tasks: 96 total, 2 running, 94 sleeping, 0 stopped, 0 zombie
%Cpu(s): 0.5 us, 0.0 sy, 0.0 ni, 99.5 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st
KiB Mem : 1865280 total, 1327824 free, 168240 used, 369216 buff/cache
KiB Swap: 2097148 total, 2097148 free, 0 used. 1493456 avail Mem
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
1867	root	20	0	162060	2200	1544	R	0.0	0.1	0:00.00	top
1866	root	10	-10	126420	1744	1352	S	0.0	0.1	0:00.00	vi
1847	root	20	0	115644	2128	1632	S	0.0	0.1	0:00.00	bash
1845	root	20	0	0	0	0	S	0.0	0.0	0:00.00	kworker/0:2
1825	root	20	0	0	0	0	R	0.0	0.0	0:00.09	kworker/0:1
1779	root	20	0	0	0	0	S	0.0	0.0	0:00.16	kworker/0:0
1665	postfix	20	0	89912	4088	3084	S	0.0	0.2	0:00.00	pickup

命令2:

```
renice -n [-20~19] [进程号]
```

作用:改变程序的运行优先级

范例:

```
renice -n 5 1866
```

```
[root@theon ~]# renice -n 5 1866
1866 (process ID) old priority -10, new priority 5
[root@theon ~]#
```

方法二: top改变进程优先级

在top命令中输入r

```
top - 17:59:44 up 2:39, 2 users, load average: 0.00, 0.01, 0.04
Tasks: 96 total, 1 running, 95 sleeping, 0 stopped, 0 zombie
%Cpu(s): 0.0 us, 0.0 sy, 0.0 ni, 100.0 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st
KiB Mem : 1865280 total, 1327948 free, 168116 used, 369216 buff/cache
KiB Swap: 2097148 total, 2097148 free, 0 used. 1493588 avail Mem
PID to renice [default pid = 1]
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
1	root	20	0	128016	6548	4152	S	0.0	0.4	0:01.20	systemd
2	root	20	0	0	0	0	S	0.0	0.0	0:00.00	kthreadd
3	root	20	0	0	0	0	S	0.0	0.0	0:00.10	ksoftirqd/0
5	root	0	-20	0	0	0	S	0.0	0.0	0:00.00	kworker/0:0H
7	root	rt	0	0	0	0	S	0.0	0.0	0:00.00	migration/0
8	root	20	0	0	0	0	S	0.0	0.0	0:00.00	rcu_bh
9	root	20	0	0	0	0	S	0.0	0.0	0:00.38	rcu_sched
10	root	0	-20	0	0	0	S	0.0	0.0	0:00.00	lru-add-drain
11	root	rt	0	0	0	0	S	0.0	0.0	0:00.03	watchdog/0
13	root	20	0	0	0	0	S	0.0	0.0	0:00.00	kdevtmpfs
14	root	0	-20	0	0	0	S	0.0	0.0	0:00.00	netns
15	root	20	0	0	0	0	S	0.0	0.0	0:00.00	khungtaskd

输入需要改变的进程号。如: 1866

```
KiB Swap: 2097148 total, 2097148 free, 0 used. 1493552 avail Mem
PID to renice [default pid = 1] 1866
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
1	root	20	0	128016	6548	4152	S	0.0	0.4	0:01.20	systemd
2	root	20	0	0	0	0	S	0.0	0.0	0:00.00	kthre
3	root	20	0	0	0	0	S	0.0	0.0	0:00.10	ksoft
5	root	0	-20	0	0	0	S	0.0	0.0	0:00.00	kwork
7	root	rt	0	0	0	0	S	0.0	0.0	0:00.00	mi

输入需要调整为的进程级别。如：6

```
KiB Swap: 2097148 total, 2097148 free, 0 used. 1493552 avail Mem
Renice PID 1866 to value 6
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
1	root	20	0	128016	6548	4152	S	0.0	0.4	0:01.20	systemd
2	root	20	0	0	0	0	S	0.0	0.0	0:00.00	kthre
3	root	20	0	0	0	0	S	0.0	0.0	0:00.10	ksoft
5	root	0	-20	0	0	0	S	0.0	0.0	0:00.00	kwork
7	root	rt	0	0	0	0	S	0.0	0.0	0:00.00	mi

top -p 1866

```
Tasks: 1 total, 0 running, 1 sleeping, 0 stopped, 0 zombie
%Cpu(s): 0.0 us, 0.0 sy, 0.0 ni, 100.0 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st
KiB Mem : 1865280 total, 1328072 free, 167992 used, 369216 buff/cache
KiB Swap: 2097148 total, 2097148 free, 0 used. 1493712 avail Mem
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
1866	root	26	6	126420	1744	1352	S	0.0	0.1	0:00.00	vi

(1.5) 进程的终止

1.5.1 kill命令

使用kill命令可用向进程发送信号来管理进程，包括终止进程。kill -l 查看可用的进程信号

```
[root@theon ~]# kill -l
```

1) SIGHUP	2) SIGINT	3) SIGQUIT	4) SIGILL	5) SIGTRAP
6) SIGABRT	7) SIGBUS	8) SIGFPE	9) SIGKILL	10) SIGUSR1
11) SIGSEGV	12) SIGUSR2	13) SIGPIPE	14) SIGALRM	15) SIGTERM
16) SIGSTKFLT	17) SIGCHLD	18) SIGCONT	19) SIGSTOP	20) SIGTSTP
21) SIGTTIN	22) SIGTTOU	23) SIGURG	24) SIGXCPU	25) SIGXFSZ
26) SIGVTALRM	27) SIGPROF	28) SIGWINCH	29) SIGIO	30) SIGPWR
31) SIGSYS	34) SIGRTMIN	35) SIGRTMIN+1	36) SIGRTMIN+2	37) SIGRTMIN+3
38) SIGRTMIN+4	39) SIGRTMIN+5	40) SIGRTMIN+6	41) SIGRTMIN+7	42) SIGRTMIN+8
43) SIGRTMIN+9	44) SIGRTMIN+10	45) SIGRTMIN+11	46) SIGRTMIN+12	47) SIGRTMIN+13
48) SIGRTMIN+14	49) SIGRTMIN+15	50) SIGRTMAX-14	51) SIGRTMAX-13	52) SIGRTMAX-12
53) SIGRTMAX-11	54) SIGRTMAX-10	55) SIGRTMAX-9	56) SIGRTMAX-8	57) SIGRTMAX-7
58) SIGRTMAX-6	59) SIGRTMAX-5	60) SIGRTMAX-4	61) SIGRTMAX-3	62) SIGRTMAX-2
63) SIGRTMAX-1	64) SIGRTMAX			

常见的信号如下图（最常用的是1,9,15）

信号代号	信号名称	说明
1	SIGHUP	该信号让进程立即关闭，然后重新读取配置文件之后重启。
2	SIGINT	程序终止信号，用于终止前台进程。相当于输出ctrl+c快捷键。
8	SIGFPE	在发生致命的算术运算错误时发出。不仅包括浮点运算错误，还包括溢出及除数为0等其它所有的算术的错误。
9	SIGKILL	用来立即结束程序的运行。本信号不能被阻塞、处理和忽略。一般用于强制终止进程。
14	SIGALRM	时钟定时信号，计算的是实际的时间或时钟时间。alarm函数使用该信号。
15	SIGTERM	正常结束进程的信号，kill命令的默认信号。有时如果进程已经发生问题，这个信号是无法正常终止进程的，我们才会尝试SIGKILL信号，也就是信号9。
18	SIGCONT	该信号可以让暂停的进程恢复执行，本信号不能被阻断。
19	SIGSTOP	该信号可以暂停前台进程，相当于输入ctrl+z快捷键。本信号不能被阻断。

命令格式

```
kill -[信号数] [进程PID号]
```

1.5.2 killall命令

命令格式

```
killall [选项][信号数] [进程名]
```

选项

-i: 交互式，询问是否要杀死某个进程

-l: 忽略进程名的大小写

1.5.3 pkill命令

命令格式

```
pkill [选项][信号数] [进程名]
```

选项

-t: 终端号，按照终端号踢出用户

范例：

```
# 查询当前有哪些用户登录
w
#last|grep "still logged in"
# 踢出用户
pkill -t -9 pts/0
```

(1.6) 进程和工作管理

1.6.1 前台、后台进程简介

Linux后台进程:

也叫守护进程 (Daemon) , 是运行在后台的一种特殊进程。

守护的意思就是不受终端控制; Linux的大多数服务就是用守护进程实现的。

比如, Web服务器httpd等。

Linux前台进程:

用户使用的有控制终端的进程.

1.6.2 前/后台进程的运行

将进程放入后台

方法一: `[命令] &`

在执行的命令后加上&符 (在后台运行)

如:

```
vim b.txt &
```

方法二: 在执行命令后加ctrl+z (在后台暂停运行)

如

```
vim b.txt  
#然后按下ctrl+z
```

1.6.3 后台进程的管理

- 查看后台进程

命令

```
jobs [-l]
```

选项:

-l: 显示工作的PID

"+" 代表最近一个放入后台的工作, 也是工作恢复时, 默认恢复的工作。 "-"号代表倒数第二个放入后台的工作

同时, 只有当前将进程放入后台的shell才能使用jobs看到后台进程信息, 切换成其他shell后执行jobs是看不到的

扩展: 执行jobs命令后, 可用使用 `kill [信号] %n` 的方式关闭后台进程, n是jobs命令输出结果中, 后台进程的工作号

- 恢复后台进程到前台

命令

```
fg %[工作号]
```

范例

```
# 进程后台执行
vim aab.txt &
# 查看后台运行的进程
jobs
# 后台进程恢复到前台
fg %1
```

- 恢复后台暂停的工作到后台继续执行

命令

```
bg %[工作号]
```

需要注意：

- 1.通常是执行了ctrl+z暂停到后台的进程才使用
- 2.此进程不能有交互界面，否则无法执行成功

二、系统资源管理

(2.1) 系统资源查看

2.1.1 监控系统资源

命令

```
vmstat [刷新时间 刷新次数]
```

```
[root@theon ~]# vmstat 1 3
procs -----memory----- --swap-- ----io---- -system-- -----cpu-----
r  b    swpd   free   buff  cache   si   so    bi    bo    in   cs  us  sy  id  wa  st
2  0      0 1493064   2108 203284    0    0   25    2   51   66  0  0 100  0  0
0  0      0 1493064   2108 203284    0    0    0    0   50   49  0  0 100  0  0
0  0      0 1493064   2108 203284    0    0    0    0   47   37  0  0 100  0  0
[root@theon ~]#
```

输出内容介绍：

1. `procs` (进程)

- **r: runnable** (可运行) 队列中的进程数量。这些进程正在运行或等待CPU运行。
- **b: blocked** (阻塞) 进程的数量。这些进程处于不可中断的睡眠状态，通常是因为等待I/O操作完成。

2. `memory` (内存)

- **swpd: swapped** (已交换) 内存量 (KB)。表示已使用的虚拟内存总量，其中包含了被换出到交换空间的内存。
- **free: free** (空闲) 内存量 (KB)。系统当前可用的空闲物理内存。
- **buff: buffers** (缓冲) 内存量 (KB)。用作文件系统缓冲的内存。
- **cache: cache** (缓存) 内存量 (KB)。用作页面缓存的内存，主要用于文件系统读写。

- **inact** (使用 `-a` 选项时显示): **inactive** (非活动) 内存量 (KB)。最近不活跃的内存，但可能包含缓存的数据。
- **active** (使用 `-a` 选项时显示): **active** (活动) 内存量 (KB)。最近活跃的内存。

3. `swap` (交换空间)

- **si: swap in** (交换入) 量 (KB/s)。每秒从磁盘交换到内存的数据量。高值可能表示内存不足。
- **so: swap out** (交换出) 量 (KB/s)。每秒从内存交换到磁盘的数据量。高值通常是内存瓶颈的明显信号。

4. `io` (I/O)

- **bi: blocks in** (块入) 量 (KB/s)。每秒从块设备（如磁盘）接收的块数量（输入）。
- **bo: blocks out** (块出) 量 (KB/s)。每秒发送到块设备的块数量（输出）。

5. `system` (系统)

- **in: interrupts** (中断) 数量。每秒钟发生的总中断次数，包括时钟中断。
- **cs: context switches** (上下文切换) 数量。每秒钟发生的上下文切换次数。高值可能表示CPU在不同进程之间频繁切换，导致效率下降。

6. `cpu` (CPU)

这些值表示CPU时间的百分比。

- **us: user** (用户) 时间百分比。CPU在用户模式下执行非内核代码所占用的时间，包括用户进程和nice进程。
- **sy: system** (系统) 时间百分比。CPU在内核模式下执行内核代码（如系统调用、I/O操作）所占用的时间。
- **id: idle** (空闲) 时间百分比。CPU处于空闲状态的时间。
- **wa: I/O wait** (I/O 等待) 时间百分比。CPU等待I/O操作完成的时间。高wa值通常表明磁盘I/O是瓶颈。
- **st: steal** (被窃取) 时间百分比。适用于虚拟化环境，表示Hypervisor从当前虚拟机“窃取”的CPU时间，通常是因为宿主机上的其他虚拟机也在争用CPU。

2.1.2 内核信息检测

查看系统启动时，内核相关信息

命令

```
dmesg
```



```
[root@theon ~]# dmesg|more
[ 0.000000] Initializing cgroup subsys cpuset
[ 0.000000] Initializing cgroup subsys cpu
[ 0.000000] Initializing cgroup subsys cpuacct
[ 0.000000] Linux version 3.10.0-862.el7.x86_64 (builder@kbuilder.dev.centos.org) (gcc version 4.8.4:24 UTC 2018)
[ 0.000000] Command line: BOOT_IMAGE=/vmlinuz-3.10.0-862.el7.x86_64 root=/dev/mapper/centos-root
p rhgb quiet LANG=en_US.UTF-8
[ 0.000000] e820: BIOS-provided physical RAM map:
[ 0.000000] BIOS-e820: [mem 0x0000000000000000-0x000000000009ebff] usable
[ 0.000000] BIOS-e820: [mem 0x000000000009ec00-0x000000000009ffff] reserved
[ 0.000000] BIOS-e820: [mem 0x00000000000dc000-0x00000000000fffff] reserved
[ 0.000000] BIOS-e820: [mem 0x0000000000100000-0x00000000007fedffff] usable
[ 0.000000] BIOS-e820: [mem 0x00000000007fee0000-0x00000000007fefefff] ACPI data
[ 0.000000] BIOS-e820: [mem 0x00000000007feff000-0x00000000007fefffff] ACPI NVS
[ 0.000000] BIOS-e820: [mem 0x00000000007ff00000-0x00000000007fffffff] usable
[ 0.000000] BIOS-e820: [mem 0x0000000000f0000000-0x0000000000f7ffffff] reserved
[ 0.000000] BIOS-e820: [mem 0x0000000000fec00000-0x0000000000fec0ffff] reserved
[ 0.000000] BIOS-e820: [mem 0x0000000000fee00000-0x0000000000fee0ffff] reserved
[ 0.000000] BIOS-e820: [mem 0x0000000000fffe0000-0x0000000000ffffffff] reserved
[ 0.000000] NX (Execute Disable) protection: active
[ 0.000000] SMBIOS 2.7 present.
[ 0.000000] DMI: VMware, Inc. VMware Virtual Platform/440BX Desktop Reference Platform, BIOS 6.00
[ 0.000000] Hypervisor detected: VMware
[ 0.000000] Host bus clock speed read from hypervisor : 66000000 Hz
[ 0.000000] e820: update [mem 0x00000000-0x00000fff] usable ==> reserved
[ 0.000000] e820: remove [mem 0x000a0000-0x000fffff] usable
[ 0.000000] e820: last_pfn = 0x80000 max_arch_pfn = 0x40000000
[ 0.000000] MTRR default type: uncachable
[ 0.000000] MTRR fixed ranges enabled:
```

2.1.3 查看内存使用状态

命令

```
free [-b|-k|-m|-g]
```

选项:

- b: 以字节为单位显示
- k: 以KB为单位显示，默认就是以KB为单位显示
- m: 以MB为单位显示
- g: 以GB为单位显示

通常以MB为单位: `free -m`

```
[root@theon ~]# free -m
              total        used        free      shared  buff/cache   available
Mem:           1821         162        1458           9          200        1462
Swap:          2047           0         2047
```

2.1.4 CPU负载情况查看

命令

```
uptime
```

显示系统的启动时间和平均负载，也就是top命令的第一行。w命令也可以看到这个数据。

2.1.5 查看系统内核

命令

```
uname -a
```

输出内容

```
[root@theon ~]# uname -a
Linux theon 3.10.0-862.el7.x86_64 #1 SMP Fri Apr 20 16:44:24 UTC 2018 x86_64 x86_64 x86_64 GNU/Linux
```

2.1.6 判断当前系统位数

参考命令

```
file /bin/ls
```

```
[root@theon ~]# file /bin/ls
/bin/ls: ELF 64-bit LSB executable, x86-64, version 1 (SYSV), dynamically linked (uses shared
cececb990b3e987, stripped
[root@theon ~]#
```

2.1.7 查询当前Linux系统的发行版本

参考命令（部分发行版有效）

```
cat /etc/*release
```

```
[root@theon ~]# cat /etc/*release
CentOS Linux release 7.5.1804 (Core)
NAME="CentOS Linux"
VERSION="7 (Core)"
ID="centos"
ID_LIKE="rhel fedora"
VERSION_ID="7"
PRETTY_NAME="CentOS Linux 7 (Core)"
ANSI_COLOR="0;31"
CPE_NAME="cpe:/o:centos:centos:7"
HOME_URL="https://www.centos.org/"
BUG_REPORT_URL="https://bugs.centos.org/"

CENTOS_MANTISBT_PROJECT="CentOS-7"
CENTOS_MANTISBT_PROJECT_VERSION="7"
REDHAT_SUPPORT_PRODUCT="centos"
REDHAT_SUPPORT_PRODUCT_VERSION="7"

CentOS Linux release 7.5.1804 (Core)
CentOS Linux release 7.5.1804 (Core)
```

2.1.8 列出进程打开或使用的文件信息

命令

`lsdf` [选项]

选项:

- c 字符串: 只列出以字符串开头的进程打开的文件
- u 用户名: 只列出某个用户的进程打开的文件
- p pid: 列出某个PID进程打开的文件

此命令可用查询某个进程是哪个二进制文件运行的

最小化安装需要执行 `yum -y install lsdf` 安装

(2.2) 系统计划任务

定时任务生效的准备-：启动crond(定时任务服务)默认启动不需要修改

```
[root@theon ~]# systemctl list-unit-files|grep enable|grep crond
crond.service                                enabled
```

2.2.1 使用crontab设置定时任务

进入定时任务编辑的命令

```
crontab -e
```

编辑规则

```
* * * * * 执行的命令
```

关于前面*****含义：

项目	含义	范围
第一个 “*”	一小时当中的第几分钟	0-59
第二个 “*”	一天当中的第几小时	0-23
第三个 “*”	一个月当中的第几天	1-31
第四个 “*”	一年当中的第几个月	1-12
第五个 “*”	一周当中的星期几	0-7（0和7都代表星期日）

特殊符号的含义

,	代表不连续的时间。比如 “0 8,12,16 * * * 命令”，就代表在每天的8点0分，12点0分，16点0分都执行一次命令
-	代表连续的时间范围。比如 “0 5 * * 1-6命令”，代表在周一到周六的凌晨5点0分执行命令
/n	代表每隔多久执行一次。比如 “/10 * * * * 命令”，代表每隔10分钟就执行一遍命令

范例：

时间	含义
45 22 * * * 命令	在22点45分执行命令
0 17 * * 1 命令	每周1 的17点0分执行命令
0 5 1,15 * * 命令	每月1号和15号的凌晨5点0分执行命令
40 4 * * 1-5 命令	每周一到周五的凌晨4点40分执行命令
*/10 4 * * * 命令	每天的凌晨4点，每隔10分钟执行一次命令
0 0 1,15 * 1 命令	每月1号和15号，每周1的0点0分都会执行命令。注意：星期几和几号最好不要同时出现，因为他们定义的都是天。非常容易让管理员混乱。

crontab 的常用选项

选项：

-e:

编辑crontab定时任务

-l:

查询crontab任务

-r:

删除当前用户所有的crontab任务

2.2.2 单次执行的定时任务

命令：

```
at [时间表达式]
at>[需要执行的命令]
at>ctrl+d 结束
```

最小化安装，需要执行 yum -y install at安装，并且执行systemctl start atd运行

关于 [时间表达式]：at支持很多时间表达式，如

now+7min :7分钟后

now+7day : 7天后

22:10 2027-08-19 : 2027年8月19日的晚上10点10分

范例：

```
at 23:44
at> echo 111
at> ctrl+d <EOT>
```

```
[root@theon ~]# at 23:44
at> echo 111
at> <EOT>
job 2 at Wed Jun 11 23:44:00 2025
[root@theon ~]#
```

查看at任务

at -l 或 atq

```
[root@theon ~]# at -l
1      Wed Jun 11 23:44:00 2025 a root
2      Wed Jun 11 23:44:00 2025 a root
3      Wed Jun 11 14:05:00 2025 a root
```

删除at任务

at -d [任务号]

三、网络管理

(3.1) 网络配置

3.1.1 临时配置

用ifconfig配置ip

如下命令：

`ifconfig ens33 172.19.205.212 255.255.255.0` 其中ens33是网卡名称，也有可能是其他，比如ens3、eth0等

扩展： 同一个网卡配置多个IP（类似路由器虚拟网卡）

`ifconfig ens33:0 192.168.3.251`


```
[root@theon ~]# ifconfig ens33:0 192.168.3.251
[root@theon ~]# ifconfig
ens33: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
    inet 192.168.239.149  netmask 255.255.255.0  broadcast 192.168.239.255
    inet6 fe80::ee98:9743:87a1:9ad1  prefixlen 64  scopeid 0x20<link>
    ether 00:0c:29:fd:5f:f9  txqueuelen 1000  (Ethernet)
    RX packets 10048  bytes 839290 (819.6 KiB)
    RX errors 0  dropped 0  overruns 0  frame 0
    TX packets 8840  bytes 1168896 (1.1 MiB)
    TX errors 0  dropped 0  overruns 0  carrier 0  collisions 0

ens33:0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
    inet 192.168.3.251  netmask 255.255.255.0  broadcast 192.168.3.255
    ether 00:0c:29:fd:5f:f9  txqueuelen 1000  (Ethernet)

lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
    inet 127.0.0.1  netmask 255.0.0.0
    inet6 ::1  prefixlen 128  scopeid 0x10<host>
    loop txqueuelen 1000  (Local Loopback)
    RX packets 2  bytes 104 (104.0 B)
    RX errors 0  dropped 0  overruns 0  frame 0
    TX packets 2  bytes 104 (104.0 B)
    TX errors 0  dropped 0  overruns 0  carrier 0  collisions 0
```

取消虚拟网卡：

```
ifconfig ens33:0 down
```

3.1.2 永久配置

修改网络配置文件

网卡信息文件目录位置： `/etc/sysconfig/network-scripts/`

```
[root@theon ~]# ls /etc/sysconfig/network-scripts/
ifcfg-ens33      ifdown-bnep  ifdown-isdn  ifdown-sit      ifup          ifup-ippv6  ifup-plusb  ifup-sit        ifup-wireless
ifcfg-ens33.bak  ifdown-eth   ifdown-post  ifdown-Team     ifup-aliases  ifup-ipv6   ifup-post   ifup-Team       init.ipv6-global
ifcfg-lo         ifdown-ippv6 ifdown-ppp    ifdown-TeamPort ifup-bnep      ifup-isdn   ifup-ppp    ifup-TeamPort   network-functions
ifdown          ifdown-ipv6  ifdown-routes ifdown-tunnel   ifup-eth       ifup-plip   ifup-routes ifup-tunnel     network-functions-ipv6
[root@theon ~]#
```

配置文件通常是 `ifcfg-[网卡名称]` 上面范例图中是ens33

```
[root@theon network-scripts]# cat ifcfg-ens33
TYPE="Ethernet"
PROXY_METHOD="none"
BROWSER_ONLY="no"
BOOTPROTO="dhcp"
DEFROUTE="yes"
IPV4_FAILURE_FATAL="no"
IPV6INIT="yes"
IPV6_AUTOCONF="yes"
IPV6_DEFROUTE="yes"
IPV6_FAILURE_FATAL="no"
IPV6_ADDR_GEN_MODE="stable-privacy"
NAME="ens33"
UUID="5ed3e42a-f5a3-4bf1-bba8-b3bfa13a4f60"
DEVICE="ens33"
ONBOOT="yes"
ZONE=public
```

配置文件参数解释

参数名	作用说明
TYPE="Ethernet"	网络接口类型为以太网（Ethernet）。
PROXY_METHOD="none"	不使用代理方法，通常与 NetworkManager 配合使用。
BROWSER_ONLY="no"	指定连接是否仅供浏览器使用，"no" 表示不只是浏览器使用。
BOOTPROTO="dhcp"	启动协议，dhcp 表示自动通过 DHCP 获取 IP 地址。
DEFROUTE="yes"	是否将该接口作为默认路由接口，"yes" 表示是。
IPV4_FAILURE_FATAL="no"	若 IPv4 配置失败是否终止连接，"no" 表示不会终止。
IPV6INIT="yes"	是否启用 IPv6 支持，"yes" 表示启用。
IPV6_AUTOCONF="yes"	是否自动配置 IPv6 地址，"yes" 表示自动配置。

参数名	作用说明
<code>IPV6_DEFROUTE="yes"</code>	是否将该接口用于 IPv6 默认路由, "yes" 表示是。
<code>IPV6_FAILURE_FATAL="no"</code>	若 IPv6 配置失败是否终止连接, "no" 表示不会终止。
<code>IPV6_ADDR_GEN_MODE="stable-privacy"</code>	IPv6 地址生成方式, "stable-privacy" 表示生成稳定但具有隐私保护的地址 (可重复, 但不泄露 MAC 地址)。
<code>NAME="ens33"</code>	该连接的名称, 通常与设备名一致。
<code>UUID="5ed3e42a-f5a3-4bf1-bba8-b3bfa13a4f60"</code>	唯一标识该连接的 UUID, 由 NetworkManager 管理生成。
<code>DEVICE="ens33"</code>	绑定的实际设备名, 通常是系统识别的网卡名。
<code>ONBOOT="yes"</code>	是否在系统启动时自动启用该连接, "yes" 表示是。
<code>ZONE=public</code>	分配给接口的 firewalld 区域, "public" 表示对外通信受限制 (常用于不信任网络)。

(3.2) 其他网络配置

3.2.1 主机名配置

使用 `hostnamectl` 目录进行管理即可

主机名配置常用命令

```
hostnamectl set-hostname [主机名]
```

3.2.2 DNS配置

文件 `/etc/resolv.conf` 管理着服务器的DNS解析查询地址

```
[root@theon network-scripts]# cat /etc/resolv.conf
# Generated by NetworkManager
search localdomain
nameserver 192.168.239.2
```

部分木马会修改这个文件, 将常用的域名地址指向恶意的IP, 以方便木马制造者进行下一步的攻击

(3.3) 常见网络管理命令

3.3.1 网卡查看

`ifconfig`

```
[root@theon network-scripts]# ifconfig
ens33: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.239.149 netmask 255.255.255.0 broadcast 192.168.239.255
    inet6 fe80::ee98:9743:87a1:9ad1 prefixlen 64 scopeid 0x20<link>
    ether 00:0c:29:fd:5f:f9 txqueuelen 1000 (Ethernet)
    RX packets 11426 bytes 942841 (920.7 KiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 10104 bytes 1269572 (1.2 MiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 6 bytes 440 (440.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 6 bytes 440 (440.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

3.3.2 网卡关闭/启动

禁用该网卡设备

`ifdown` [网卡设备名]

启用该网卡设备

`ifup` [网卡设备名]

3.3.3 查询网络状态

命令格式

`netstat` [选项]

选项:

- t: 列出TCP协议端口
- u: 列出UDP协议端口
- n: 不使用域名与服务名, 而使用IP地址和端口号
- l: 仅列出在监听状态网络服务
- a: 列出所有的网络连接

常用查询如下

- `netstat -tulnp` 查询所有启动网络监听的服务

t----->tcp协议
u----->udp协议
l----->监听
n----->使用ip和端口号
p----->进程号

- `netstat -an` 查询所有的网络连接

3.3.4 查询路由

查看操作系统本地的路由表

```
route -n
```

```
[root@theon network-scripts]# route -n
Kernel IP routing table
Destination      Gateway          Genmask         Flags Metric Ref    Use Iface
0.0.0.0          192.168.239.2   0.0.0.0         UG      100    0      0 ens33
192.168.239.0    0.0.0.0         255.255.255.0   U       100    0      0 ens33
```

3.3.5 路由追踪命令

命令格式

```
tracert [选项] IP或域名
```

选项:

-n 使用IP，不使用域名，速度更快

建议添加上 `-I` 参数在进行路由追踪，原因如下图：

使用UDP协议探测，路由节点不给IP信息

使用ICMP协议探测，路由节点返回IP

四、其他



IT入门基础|网络安全运维知识

400



下载

订阅

分享

将节目插入微信公众号，（公众号菜单或文章中），增加曝光和播放量~

操作指南 >

✕

零基础也能学的 IT 运维 + 网络安全入门，从常用软件、网络基础、Windows/Linux 命令学起，一步步掌握 Docker 虚拟化、Python 爬虫、Web 中间件配置，再深入渗透测试框架、PHP 漏洞分析等实战内容。全程大白话讲解，不堆砌晦涩术语，不用啃厚书、不费眼刷视频，走路、睡前、通勤时听一听，就能轻松入门 IT 运维与网络安全！

声音 (13) 评价 (0)

正序 | 倒序

13 Linux服务器底层生存法则

0

13分钟前

播客标题：Linux服务器底层生存法则

播客内容：深入浅出的探讨 Linux 的系统管理包括系统管理核心操作、底层原理

播客地址：

- 喜马拉雅：<https://www.ximalaya.com/album/118679212>
- 小宇宙：<https://namecard.xiaoyuzhoufm.com/rjrtv>

目的：辅助掌握今天的知识内容，没有时间学习的同学或者已经有基础的同学可以通过播客快速掌握大致内容

作业

情景作业：服务器资源监测

任务描述：作为一个企业的安全运维工程师，监控服务器的资源使用状态是你常态化工作之一，请编写一个shell脚本，脚本的功能如下：

- 每隔30秒，输出当前操作系统的内存使用单位为MB、内存剩余（单位为MB）、CPU使用率、IP信息、操作系统发行版本
- 第一行 输出格式为：当前系统时间为 年-月-日-时-分-秒 系统网络IP地址为 xx.xx.xx.xx，发行版本 xxxx
- 第二行 输出格式为：系统CPU使用率为 x%，内存使用为 n MB，剩余内存 n MB

提示：时间输出格式可用参考命令 `date +%Y-%m-%d-%H-%M-%S`

提示2：sleep [秒数] 命令可以让系统进行等待

任务提交：shell脚本输出截图（至少有三次输出结果），如下范例：

```
[root@theon ~]# sh work_20250612.sh
当前系统时间为 2025-06-11-15-55-09 系统网络IP地址为 192.168.239.149 , 发行版本 CentOS Linux release 7.5.1804
系统CPU使用率为 0% , 内存使用为 165MB , 剩余内存 1288MB
当前系统时间为 2025-06-11-15-55-39 系统网络IP地址为 192.168.239.149 , 发行版本 CentOS Linux release 7.5.1804
系统CPU使用率为 11.8% , 内存使用为 244MB , 剩余内存 953MB
当前系统时间为 2025-06-11-15-56-09 系统网络IP地址为 192.168.239.149 , 发行版本 CentOS Linux release 7.5.1804
系统CPU使用率为 6.2% , 内存使用为 182MB , 剩余内存 998MB
```